



INDIAN INSTITUTE OF
INFORMATION
TECHNOLOGY

Cloud Platforms and Frameworks

Dr. Animesh Chaturvedi

Assistant Professor: **IIIT Dharwad**

Young Researcher: **Heidelberg Laureate Forum**
and **Pingala Interaction in Computing**

Young Scientist: **Lindau Nobel Laureate Meetings**

Postdoc: **King's College London & The Alan Turing Institute**

PhD: **IIT Indore** MTech: **IIITDM Jabalpur**



IaaS and Hybrid Cloud

IaaS and Hybrid Cloud

A. Orchestration and Virtualization

A. Orchestration

B. Virtual machines

A. Type-1, native or bare-metal hypervisors

B. Type-2 or hosted hypervisors

C. Both Type 1 and Type 2

B. Content Delivery Network, or Content Distribution Network (CDN)

Infrastructure as a Service (IaaS)

- IaaS are online services
 - provide high-level APIs used to dereference (or abstract) various low-level details of infrastructure like
 - physical computing resources,
 - location,
 - data partitioning,
 - scaling,
 - security,
 - backup
- IaaS deals with Cloud Orchestration and Virtualization

Examples of IaaS Clouds

IaaS involves the use of a Cloud Orchestration and Virtualization Technologies like

- Eucalyptus
 - Hypervisors (KVM, Xen, VMware)
 - Private and
 - Hybrid cloud computing with Amazon Elastic Compute Cloud (EC2)
- Open Stack,
 - OpenStack-native REST API and
 - Hybrid cloud computing with CloudFormation-compatible Query API
- Apache Cloudstack
 - virtualization, such as KVM, VMware vSphere, including ESXi and vCenter, and XenServer/XCP.
- OpenNebula.
 - Hypervisors (VMware vCenter, KVM, LXD and AWS Firecracker)

Examples of IaaS Clouds

- [Amazon Web Services](#)
- [AppScale](#)
- [Box](#)
- [Bluemix](#)
- [CloudBolt](#)
- [Cloud Foundry](#)
- [Cocaine \(PaaS\)](#)
- [Creatio](#)
- [Engine Yard](#)
- [GreenQcloud](#)
- [IBM Cloud](#)
- [iland](#)
- [Joyent](#)
- [Linode](#)
- [Lunacloud](#)
- [Microsoft Azure](#)
- [Mirantis](#)
- [Netlify](#)
- [Nimbula](#)
- [Nimbus](#)
- [OpenIO](#)
- [OpenNebula](#)
- [OpenStack](#)
- [Oracle Cloud](#)
- [OrionVM](#)
- [Rackspace Cloud](#)
- [Safe Swiss Cloud](#)
- [SoftLayer](#)
- [Zadara Storage](#)
- [libvirt](#)
- [libguestfs](#)
- [OVirt](#)
- [Virtual Machine Manager](#)
- [Wakame-vdc](#)
- [Virtual Private Cloud OnDemand](#)

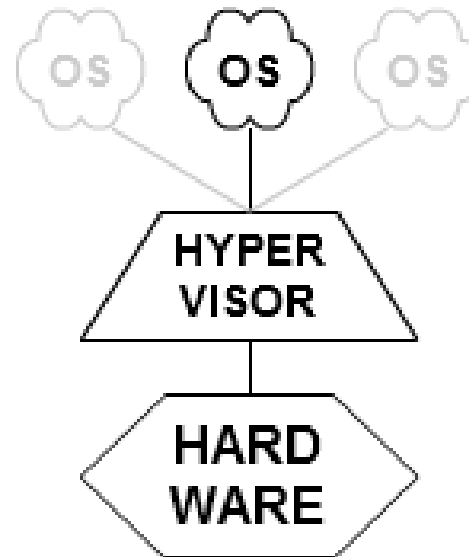
Orchestration

- Automated configuration, coordination, and management of
 - service-oriented architecture,
 - virtualization,
 - provisioning,
 - converged infrastructure and
 - dynamic datacenter.
- To meet requirements of Applications, Data, and Infrastructure.
- Difference Between
 - Workflows Automation are processed and completed as processes within a single domain for automation purposes.
 - Orchestration includes a workflow and provides a directed action towards larger goals and objectives.

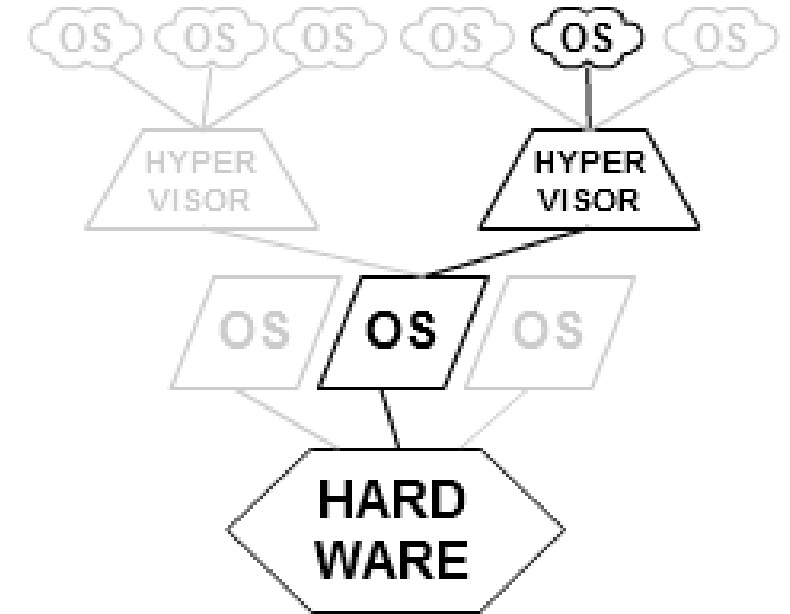
Virtualization

Hypervisor: Runs one or more virtual machines is called a *host machine*, and each virtual machine is called a *guest machine*.

- Type-1, native or bare-metal hypervisors
- Type-2 or hosted hypervisors:



TYPE 1
native
(bare metal)

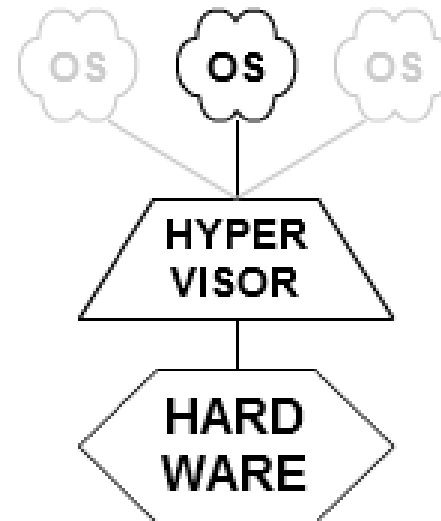


TYPE 2
hosted

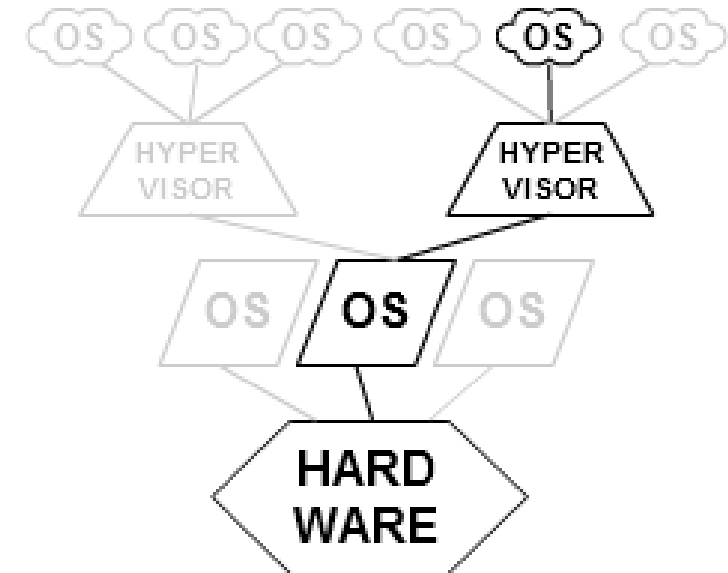
Type-1, native or bare-metal hypervisors:

run directly on the host's hardware to control the hardware and to manage guest operating systems.

- [SIMMON](#) and IBM's [CP-40](#) (first Hypervisors late 1960s)
- Microsoft [Hyper-V](#) and [Xbox One system software](#)
- [VMware ESXi](#) (formerly ESX),
- Nutanix [AHV](#),
- [XCP-ng](#),
- [Oracle VM Server for SPARC](#) and [x86](#),
- [POWER Hypervisor](#), and
- [Xen](#).



TYPE 1
native
(bare metal)

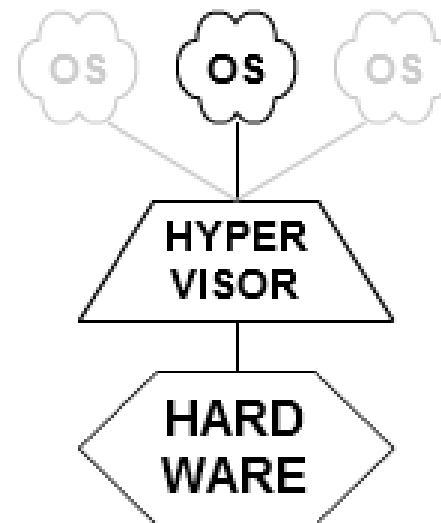


TYPE 2
hosted

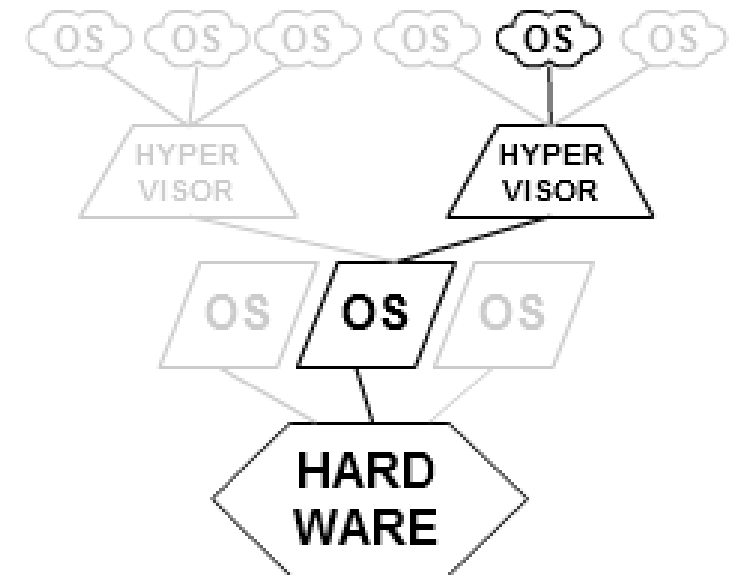
Type-2 or hosted hypervisors:

run on a conventional Operating-system of a host machine like a process as a guest machine

- [Oracle VirtualBox](#),
- [Parallels Desktop for Mac](#),
- [QEMU](#),
- [VirtualBox](#),
- [VMware Player](#) and
- [VMware Workstation](#)



TYPE 1
native
(bare metal)

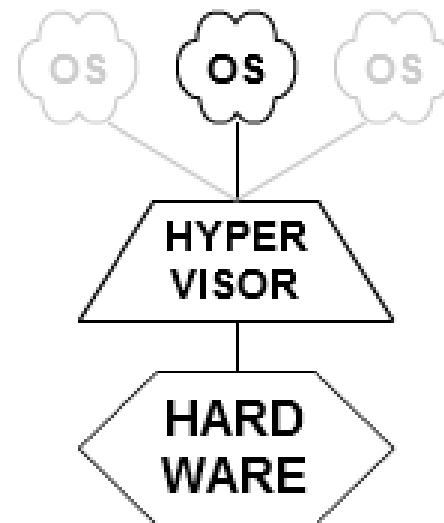


TYPE 2
hosted

Both Type 1 and Type 2

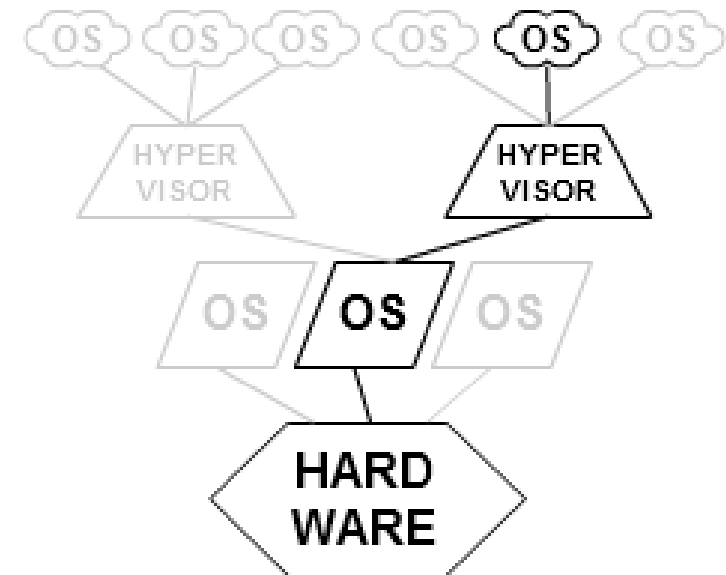
are kernel modules

- Linux's KVM (Kernel-based Virtual Machine)
- FreeBSD's bhyve (Berkeley Software Distribution BSD)



TYPE 1

native
(bare metal)



TYPE 2

hosted

IaaS and Hybrid Cloud

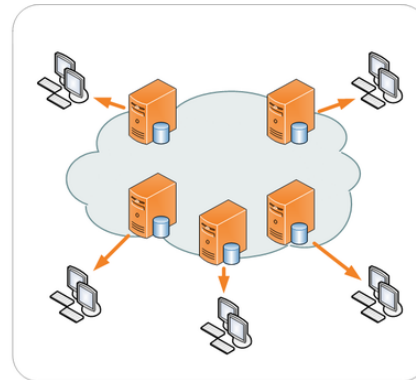
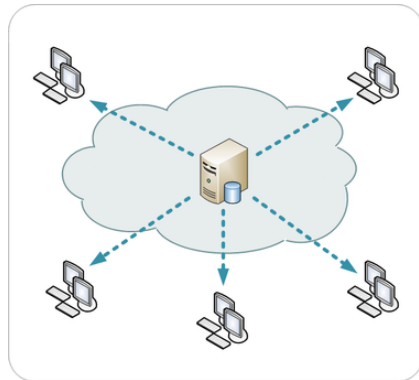
A. Service Orchestration and Virtualization

- Abstraction layer or abstraction level
- Virtual machines
- Type-1, native or bare-metal hypervisors
- Type-2 or hosted hypervisors
- Both Type 1 and Type 2

B. Content Delivery Network, or Content Distribution Network (CDN)

Content Delivery Network, or Content Distribution Network (CDN)

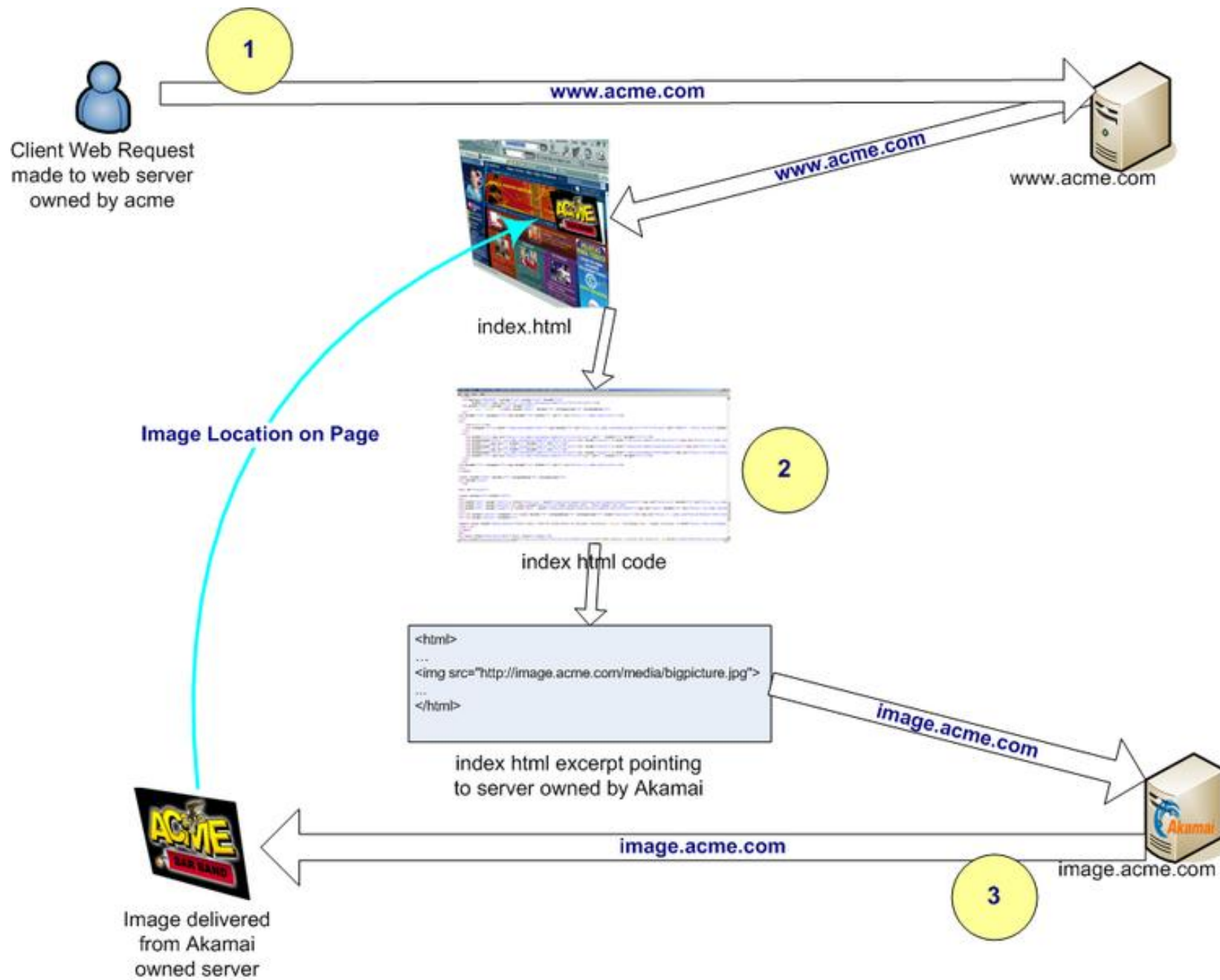
- CDN is a geographically distributed network of proxy servers and their data centers
 - to provide high availability and performance by distributing the service spatially relative to end users.
 - came in late 1990s to improve the performance bottlenecks of the Internet
- Peer-to-peer CDNs: Consumers provide resources as well as use them. Different from client-server
- Private CDNs: Content owners create their own CDN.



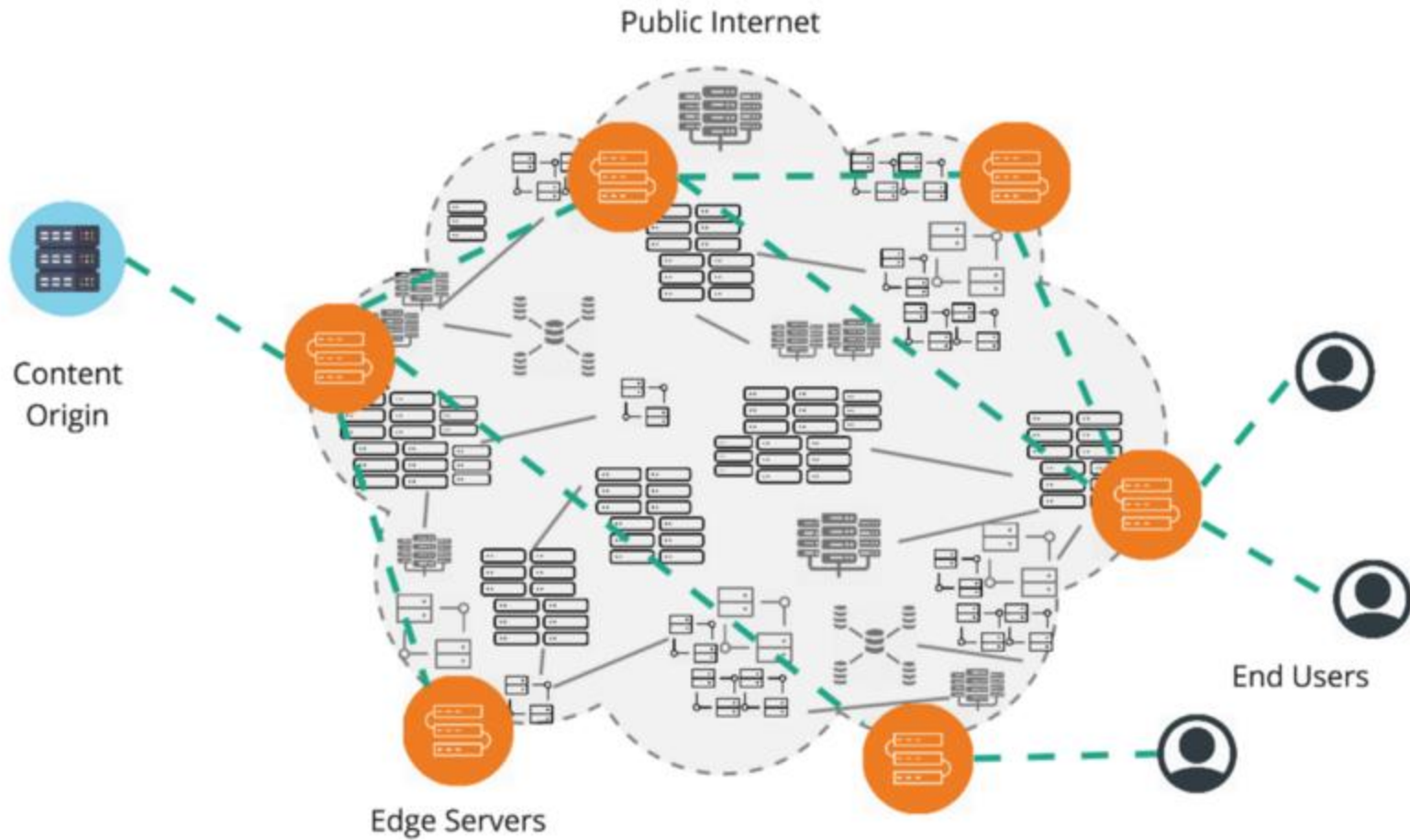
(Left) Single server distribution
(Right) CDN scheme of distribution

Akamai CDN

- Global CDN, Cybersecurity, and Cloud service provider.
- Akamai CDN is one of the largest distributed computing platform
 - Serving between 15% and 30% of all web traffic
 - Operates and rents worldwide network of servers
 - Consumer websites work faster by distributing content from locations near the end-user.
 - End-user navigates to the URL of an Akamai's consumer; their browser is redirected to one of Akamai's copies of the website.
- Facebook as Consumer of Akamai CDN uses many edge servers closer to FB consumer (i.e. FB users).



Akamai Content Delivery Network

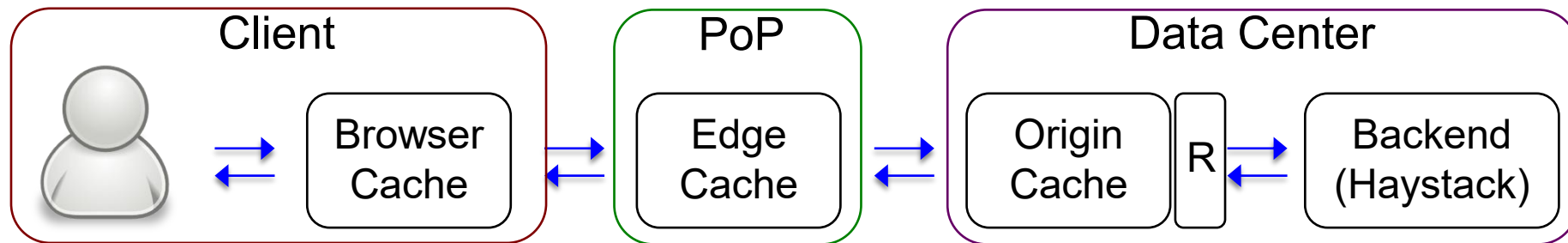


Hybrid Akamai and Facebook

- Facebook Photo Serving Stack: components are linked to show the photo retrieval work-flow.
- Desktop and Mobile clients initiate request traffic, which routes either directly to the Facebook Edge or via Akamai CDN depending on the fetch path.
- The Origin Cache collects traffic from both paths, serving images from its cache and resizing them if needed.
- The Haystack backend holds the actual image content.

Facebook's Photo-Serving Stack

- **Browser:** The first cache layer is in the client's browser
- **Edge:** The Facebook Edge is comprised of a set of Edge Caches that each run inside points of presence (PoPs) close to end users.
- **Haystack:** The backend layer accessed when there is a miss in the Origin cache (co-located with storage servers), the image can often be retrieved from a local Haystack server.

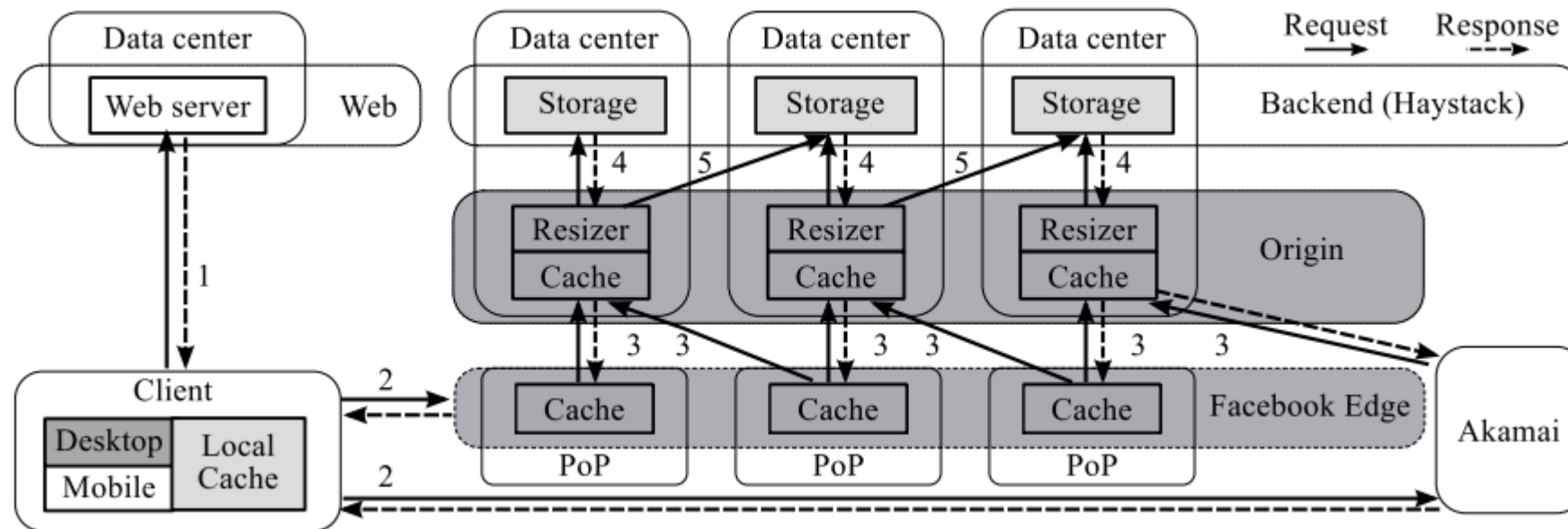


Huang, Qi, et al. "An analysis of Facebook photo caching." *Proceedings of the Twenty-Fourth ACM Symposium on Operating Systems Principles*. 2013.

Hybrid Akamai and Facebook

Architecture and full life cycle of a photo request

- Photo Transformations: Resizers transform photos that are requested by the Akamai CDN,
- Caching Stack



Huang, Qi, et al. "An analysis of Facebook photo caching." *Proceedings of the Twenty-Fourth ACM Symposium on Operating Systems Principles*. 2013.

PaaS and Container as a Service (CaaS)

Platform as a service (PaaS)

- PaaS allow customers to do application
 - development,
 - execution, and
 - management
- PaaS removes the complexity of building and maintaining the infrastructure for app
 - development and
 - deployment

Examples of PaaS Clouds

- [Amazon Web Services](#)
- [Abiquo Enterprise Edition](#)
- [CloudStack](#)
- [Citrix Cloud](#)
- [CtrlS](#)
- [DigitalOcean](#)
- [EMC Atmos](#)
- [Eucalyptus](#)
- [Fujitsu](#)
- [GoGrid](#)
- [Google Cloud Platform](#)
- [GreenButton](#)
- [Helion](#)
- [GE Predix](#)
- [Google App Engine](#)
- [GreenCloud](#)
- [Heroku](#)
- [IBM Cloud](#)
- [Inktank](#)
- [Jelastic](#)
- [Mendix](#)
- [Microsoft Azure](#)
- [MindSphere](#)
- [Netlify](#)
- [Oracle Cloud](#)
- [OutSystems](#)
- [openQRM](#)
- [OpenShift](#)
- [PythonAnywhere](#)
- [RightScale](#)
- [Scalr](#)
- [Force.com](#)
- [SAP Cloud Platform](#)
- [VMware vCloud Air](#)
- [WaveMaker](#)

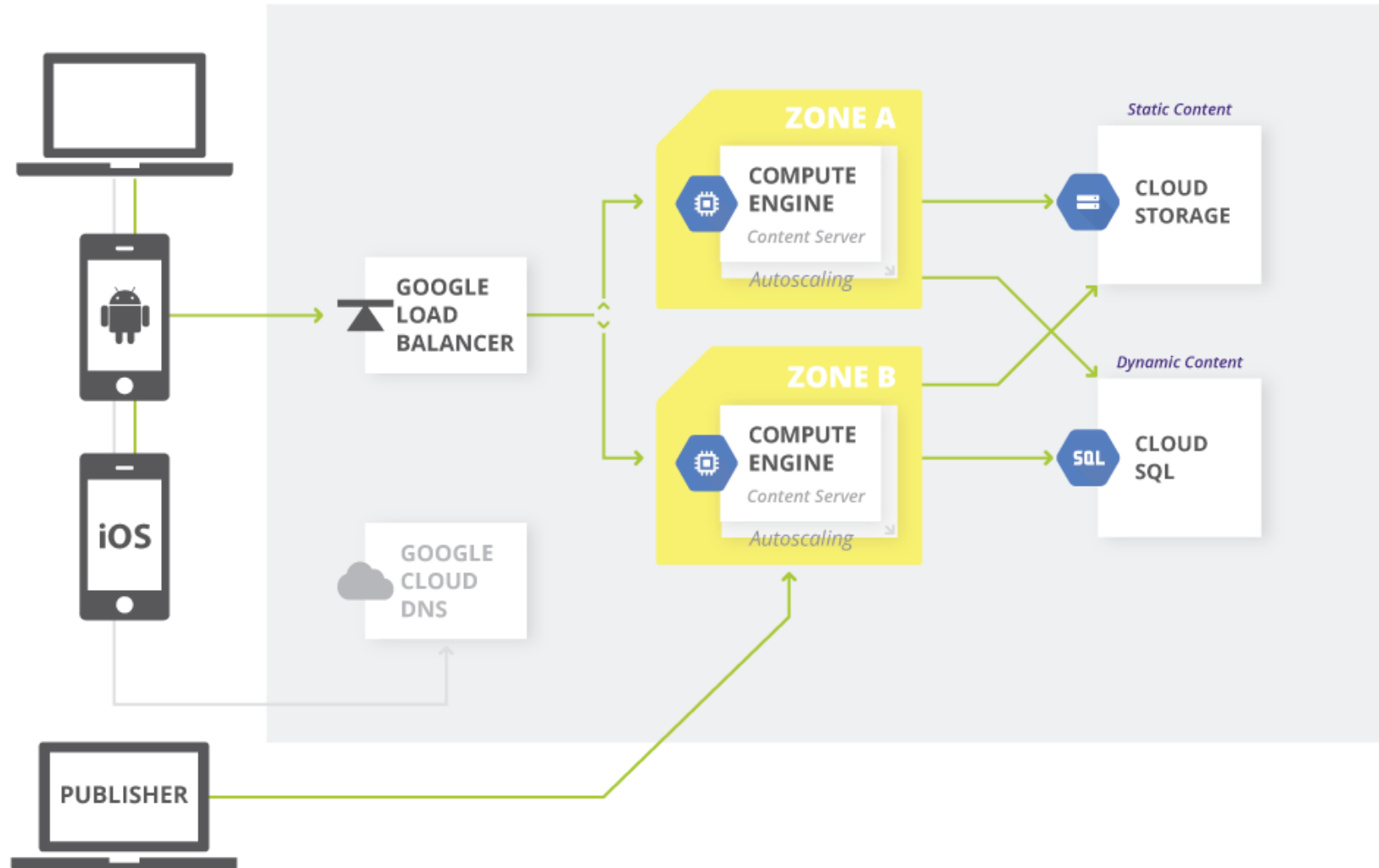
PaaS and Container as a Service (CaaS)

- A. **PaaS – Google App Engine (GAE)** and Ruby on Rails
- B. Container as a Service (CaaS) - DockerHub

Google Content Management

- Web information, marketing campaigns or social media.
- Personalized for individual users or groups.
- Google Cloud Platform (GCP) components and services to create a Content Management system.
- Google Load Balancer to support traffic routed to multiple zones for high availability.
- Google's Cloud DNS provides a robust DNS manages the domain.
- Static content → Cloud Storage
- Dynamic content → Cloud SQL implementation.

Google Content Management



<https://cloud.google.com/solutions/architecture/contentmanagement>

Google Cloud Website Hosting (Architecture)

- Google Cloud provides multiple architectures for hosting modern websites.
- **Core Hosting Technologies**
 - **Static Hosting:** Cloud Storage, Firebase Hosting
 - **Virtual Machines:** Compute Engine (IaaS)
 - **Containers:** Google Kubernetes Engine (GKE)
 - **Serverless:** Cloud Run
- **Supporting Infrastructure**
 - **Storage:** Cloud SQL, Firestore, Bigtable, Cloud Storage
 - **Networking:** HTTP(S) Load Balancer, Cloud CDN
 - **Scaling:** Managed Instance Groups, Cluster Autoscaler
 - **Monitoring:** Cloud Logging, Cloud Monitoring
- **Architecture Flow**

User → DNS → Load Balancer → Compute/Containers/Serverless → Storage

Google Cloud Website Hosting (Business)

- **Business Benefits**
 - **Scalability:** Automatically scale infrastructure during high traffic
 - **Global Performance:** Google's global data centers and CDN reduce latency
 - **Cost Efficiency:** Pay-as-you-use pricing model
 - **Reliability:** High availability and fault tolerance
 - **Security:** Built-in HTTPS, IAM, and security policies
- **Operational Advantages**
 - Faster deployment using **Cloud Marketplace**
 - Integration with **DevOps tools (CI/CD, Jenkins, Cloud Build)**
 - Supports **hybrid and multi-cloud architectures**

Google Cloud Website Hosting (Comparison)

Hosting Model	Product	Control Level	Scalability	Use Case
Static Hosting	Cloud Storage / Firebase	Low	Automatic	Blogs, documentation
Virtual Machines	Compute Engine	High	Managed instance groups	Custom web servers
Containers	GKE	Medium–High	Kubernetes autoscaling	Microservices apps
Serverless	Cloud Run	Low	Fully automatic	APIs, modern web apps

Key Insight

More control → more management responsibility

Less control → more automation

Google Cloud Website Hosting (Examples)

- **Static Website Hosting**
 - Personal or documentation websites hosted using **Cloud Storage** or **Firebase Hosting**.
Example sites: **Portfolio sites, blogs, documentation pages**.
- **Dynamic Web Application**
 - **WordPress** website hosted on **Compute Engine** with **Cloud SQL** database.
- **Container-Based Large-Scale Platforms**
 - Large services like **YouTube** run on container-based distributed infrastructure similar to **Google Kubernetes Engine (GKE)** for managing large-scale microservices.
- **Serverless Web Applications**
 - APIs or backend services similar to components used in **Gmail** or **Google Drive**, which rely on scalable cloud services and serverless architectures.
- **Global Cloud Infrastructure Example**
 - Major services like **Google Search, YouTube, and Gmail** run on the same global infrastructure that powers **Google Cloud**.

Logging and monitoring with Compute Engine

Operations
Logging

Logs Explorer

Logs Dashboard

Logs-based Metrics

Logs Router

Logs Storage

Log fields

Search fields and values

RESOURCE TYPE

Cloud HTTP Load Balancer 1

SEVERITY

Warning Clear

Query preview

severity=WARNING

Save Stream logs Run Query

Histogram

Nov 29, 6:25 PM 7:00 PM Nov 29, 7:26 PM

Query results

Jump to Now Actions Configure

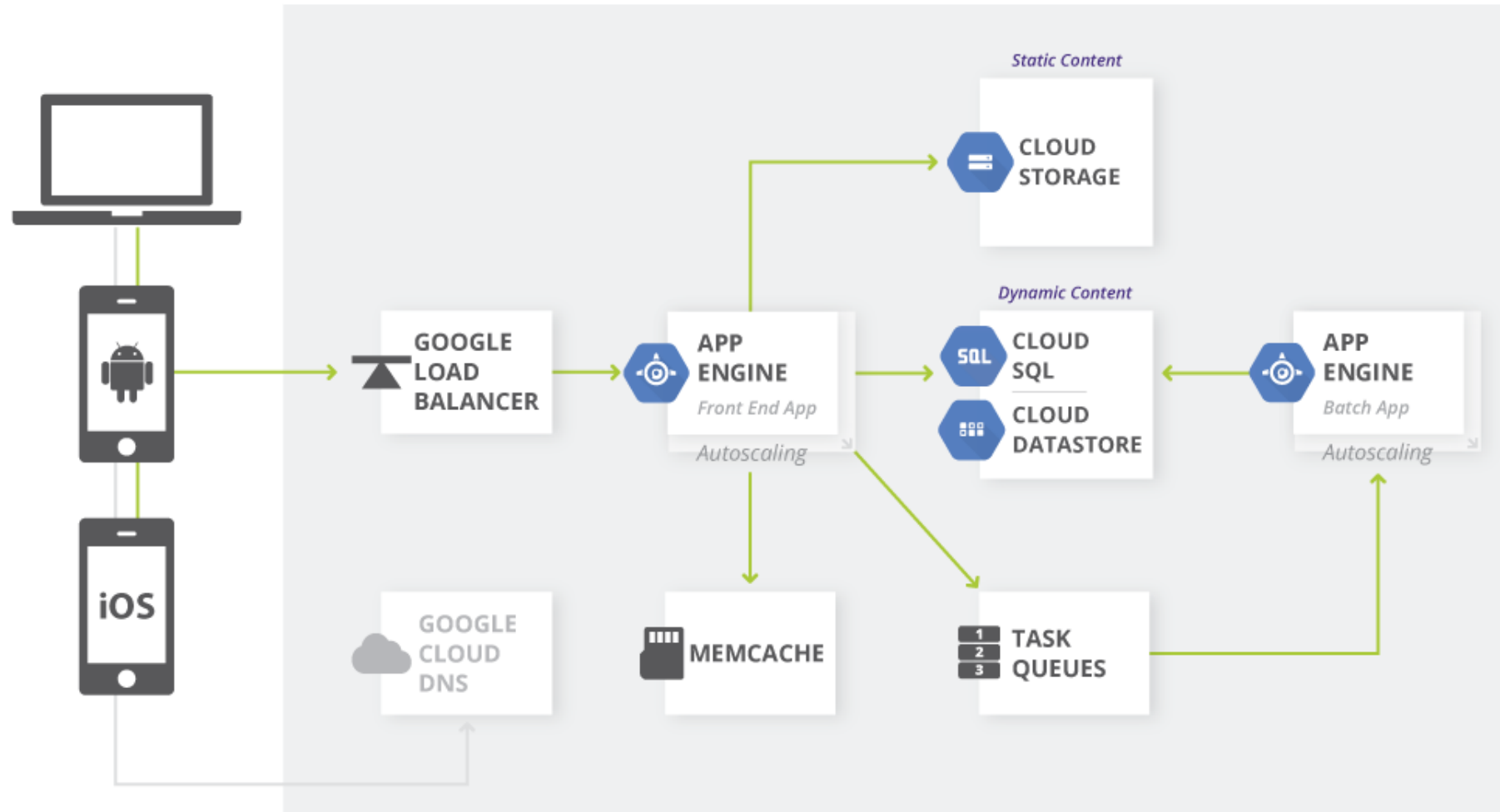
SEVERITY	TIMESTAMP	PST	SUMMARY
	Showing logs for last 1 hour starting at 11/29/20, 6:27 PM. Extend time by: 1 hour Edit time		
	2020-11-29 18:34:28.796 PST	GET 403	469 B 176 ms Mozilla/5.0 ... https://34.102.253.89/actuator/health
Hide log summary Expand nested fields Copy to clipboard Copy link <pre>{ "insertId": "1nxt148geig5040", "jsonPayload": { "cacheId": "SF0-1d5601d9", "statusDetails": "response_sent_by_backend", "@type": "type.googleapis.com/google.cloud.loadbalancing.type.LoadBalancerLogEntry" }, "httpRequest": { "requestMethod": "GET", "requestUrl": "https://34.102.253.89/actuator/health", "requestSize": "125", "status": 403, "responseSize": "469", "userAgent": "Mozilla/5.0 zgrab/0.x", "remoteIp": "192.241.222.11", "cacheLookup": true, "latency": "0.176694s" }, "resource": { "type": "http_load_balancer", "labels": {} }}</pre>			

<https://cloud.google.com/solutions/architecture/webapp>

Architecture: Web Application on Google App Engine

- Simple development and deployment of Web Applications with autoscaling compute power as well as the integrated features like distributed in-memory cache, task queues and datastore, to create robust applications quickly and easily.
- For applications written in Java, Python, PHP and Go.
- Supports multiple application versions which support A/B testing.
- Memcache is an in-memory cache to provide extremely high speed access to information cached by the web server (e.g. authentication or account information).
- Task Queues provide a mechanism to offload longer running tasks to backend servers, freeing the front end servers to service new user requests.
- Google Load Balancer which provides transparent load balancing to applications.
- Google's Cloud DNS is used to manage DNS domain of user.

Architecture: Web Application on Google



<https://cloud.google.com/solutions/architecture/webapp>

Domain Name and DNS Configuration

- Before hosting a website, a **domain name must be configured** to route users to the cloud infrastructure.
- **Domain Registration:** Register a domain using a public domain registrar. Example: example.com
- **DNS Configuration:** DNS records map domain names to servers.
- Example configuration:
 - **A Record:** example.com → IP Address
 - **CNAME Record:** www.example.com → example.com
- **DNS Service in Google Cloud:** DNS can be managed using **Google Cloud DNS**.
- Key features:
 - Managed DNS infrastructure
 - High availability
 - Low-latency DNS resolution
- Note: DNS changes propagate depending on **TTL (Time To Live)** values.

Static Website Hosting Workflow

- A static website can be hosted using
 - **Google Cloud Storage** or **Firebase Hosting**.
- **Steps to Deploy a Static Website**
 1. Create a **Cloud Storage bucket**
 2. Upload HTML, CSS, and JavaScript files
 3. Configure bucket for web hosting
 4. Connect the domain name
 5. Enable HTTPS using load balancing or Firebase
- **Static Site Development Tools**
 - Common static-site generators: **Jekyll, Hugo, Ghost**
 - These tools generate static pages from **Markdown templates**.

DevOps, Monitoring and Autoscaling

- Large-scale websites require automated **deployment and monitoring**.
- **DevOps Deployment:** Websites can be deployed using CI/CD tools such as:
 - **Jenkins, Terraform, Google Cloud Build**
 - These tools automate infrastructure creation and deployment.
- **Monitoring and Logging:** Google Cloud provides built-in observability tools:
 - **Google Cloud Logging:** Collects application and infrastructure logs.
 - **Google Cloud Monitoring:** Provides dashboards, alerts, and uptime checks.
- **Autoscaling:** Infrastructure automatically scales using:
 - Managed Instance Groups and Autoscaler policies
- **Benefits:** Handles traffic spikes; Reduces infrastructure costs during low demand

GWT to Modern Web Application Frameworks

- **Earlier Framework: Google Web Toolkit (GWT)**
 - GWT was an early framework for building **Rich Internet Applications (RIA)**.
 - A development environment similar to Web-Server or Desktop app development environment.
- **Key characteristics:**
 - Developers could build web applications using **Java instead of JavaScript**
 - Java code was **compiled into optimized JavaScript**
 - Provided tools for **debugging, refactoring, and unit testing**
 - Offered **Hosted Mode** for debugging and **Web Mode** for running compiled applications
 - Development model was similar to **desktop frameworks like Java Swing**
- **Some Google products previously used GWT components:**
 - **Google Docs**
 - **Google AdSense**
 - **Google Wallet**

Modern Web Development Technologies

- Today, rich web applications are built using **JavaScript frameworks and cloud-native architectures**.
- Popular frontend frameworks:
 - **React**
 - **Angular**
 - **Vue.js**
- Backend and API platforms:
 - **Node.js**
 - **Spring Boot**
 - **Django**
- Key modern capabilities:
 - Component-based UI design
 - REST and GraphQL APIs
 - Real-time web applications
 - Integration with cloud services

Cloud-Native Web Applications

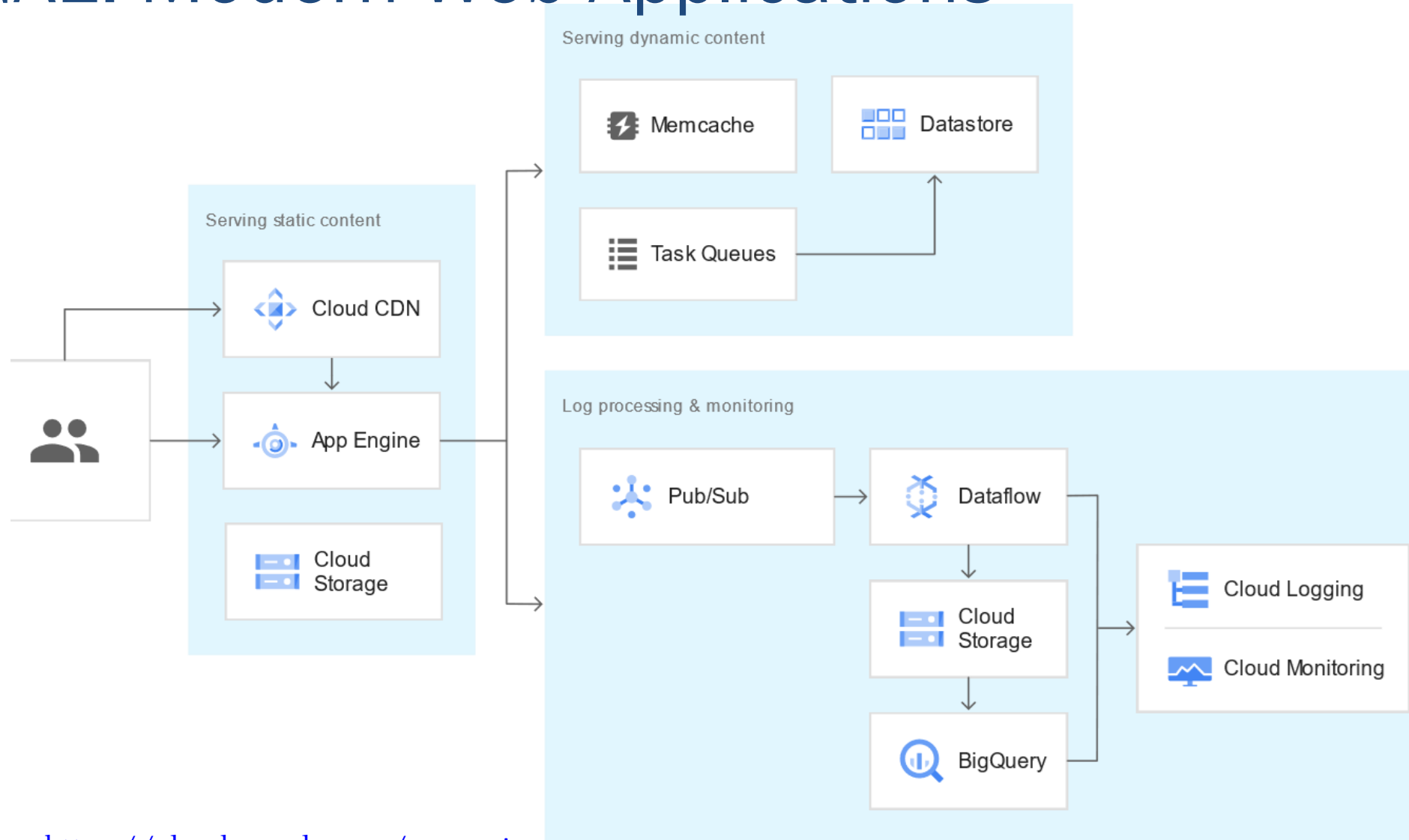
- Modern applications are commonly deployed using cloud platforms such as:
 - **Google Cloud**
 - **Amazon Web Services**
 - **Microsoft Azure**
- Typical infrastructure:
 - **Containers (Docker + Kubernetes)**
 - **Serverless computing**
 - **Microservices architecture**
 - **Continuous Integration / Continuous Deployment (CI/CD)**
- **Key Insight:** Early frameworks like **Google Web Toolkit** simplified web development using Java. Modern web development relies on **JavaScript frameworks, APIs, and scalable cloud platforms** to build interactive and globally distributed applications.

Modern Web Applications

Web app using Google Cloud.

- Quickly reach customers and end users by deploying web apps.
- Zero-config deployments and zero server management.
- Only writing code for App Engine.
- App Engine automatically scales to support sudden traffic spikes without provisioning, patching, or monitoring.

GAE: Modern Web Applications

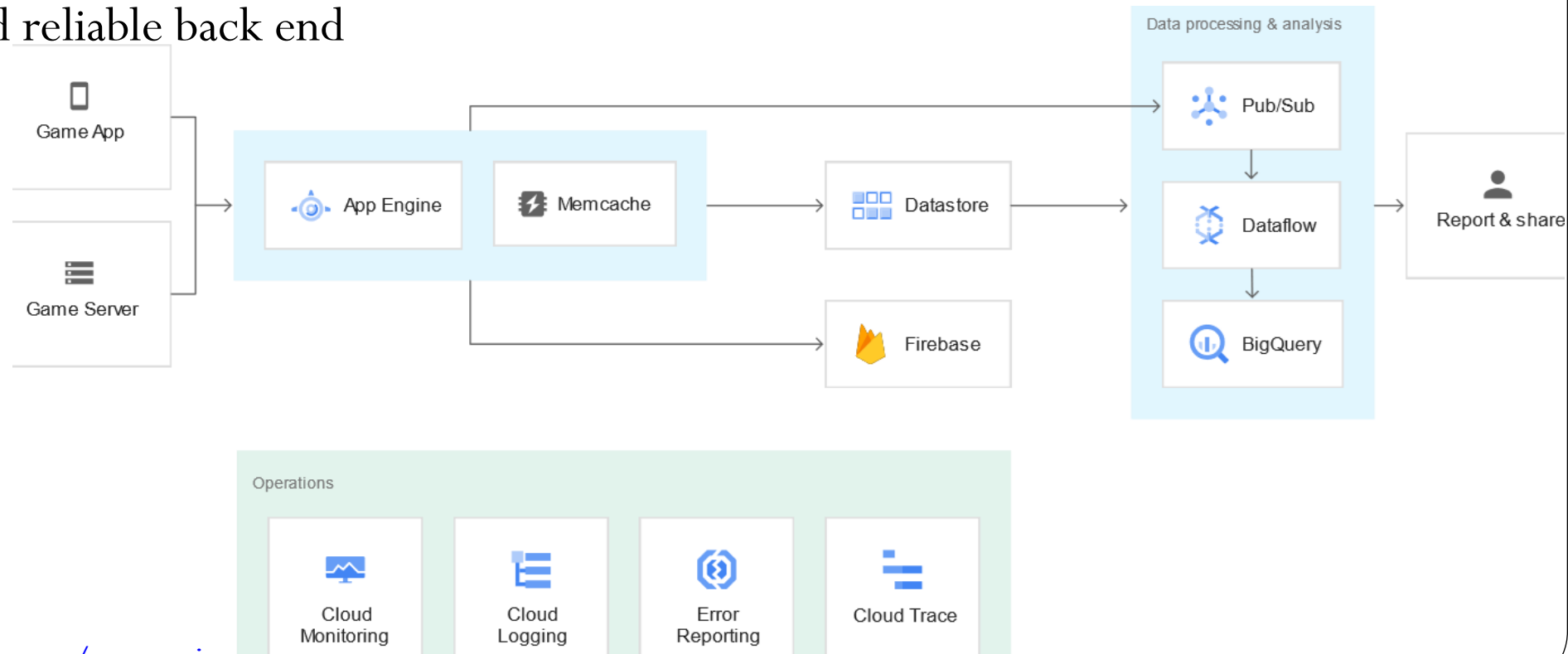


<https://cloud.google.com/appengine>

GAE: Scalable Mobile Back-ends

Mobile app built with Firebase and Google services.

- automatically scales the hosting environment.
- an easy-to-use frontend mobile platform
- scalable and reliable back end

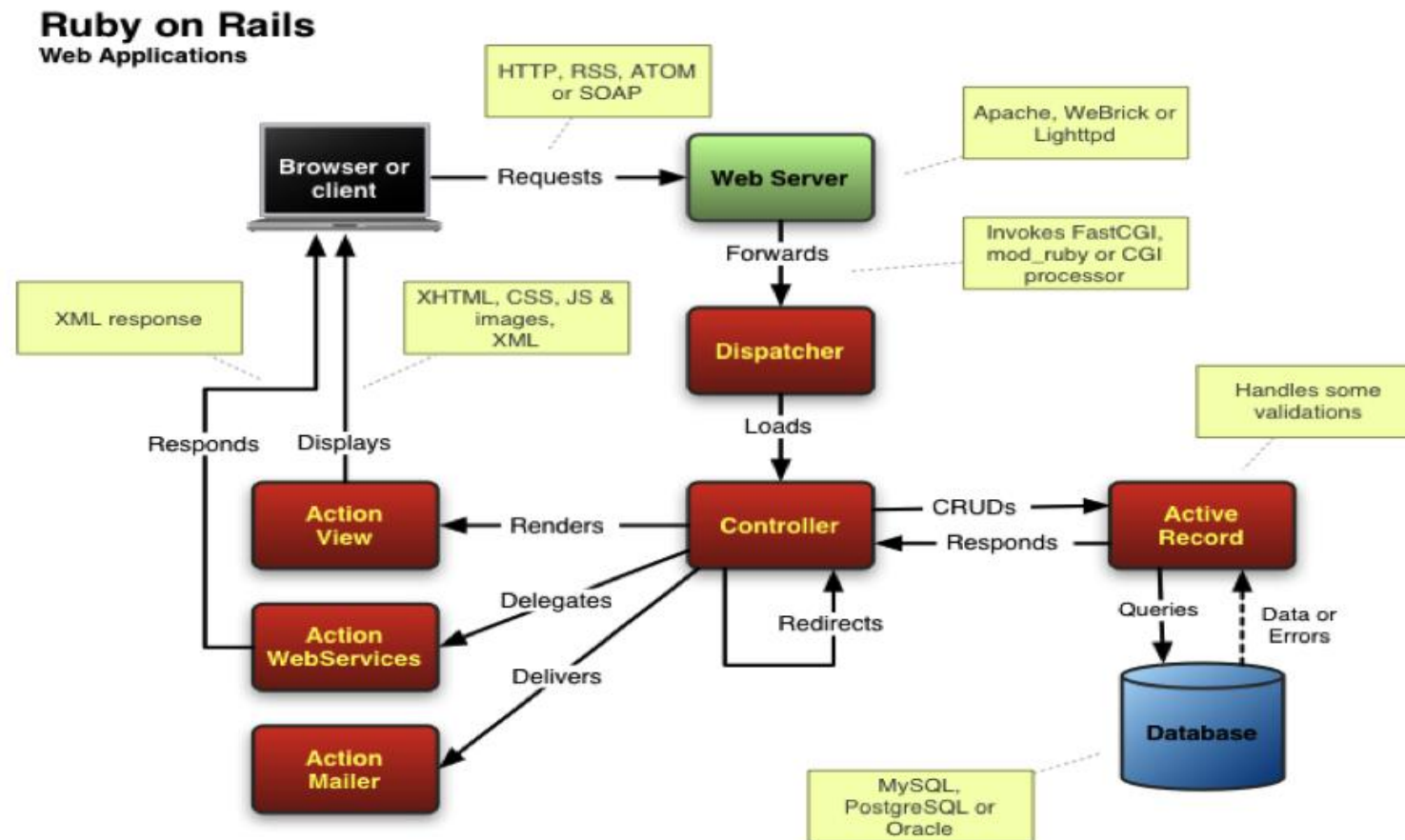


PaaS and Container as a Service (CaaS)

1. PaaS – Google App Engine (GAE) and **Ruby on Rails**
2. Container as a Service (CaaS) – DockerHub

Ruby on Rails

- A web applications development framework. Very popular in the agile development with the help of convention over configuration management. Ruby on Rails uses the Model-View-Controller (MVC)



Ruby on Rails

RESTful Architecture

- Representational State Transfer (REST) is an alternative to web services, such as SOAP and WSDL.
- Works on HTTP protocol for operations: Create, Read, Update and Delete (CRUD).
- RESTful is useful when it is required stateless, limited bandwidth (specially for mobile devices no overhead of SOAP), and when service provider and user have complete information of operations.

Ruby on Rails

- MVC separates business logic from HTML views. Architectural pattern in order to improve the maintainability of the application.
- **Model:** Carries the business logic and rules to manipulate the data.
 - Models represent the information in the database.
 - Manages interaction with database.
- **View:** Front-end of the application, representing the user interface.
 - HTML files with embedded Ruby code. Used to display data in the form of views
 - Formats, such as HTML, PDF, XML, RSS and more.
- **Controller:** it interact with models and views.
 - Incoming requests are processed by the controllers.
 - Controller process the data from the models and pass it to the views for presentation.

PaaS and Container as a Service (CaaS)

1. PaaS – Google App Engine (GAE) and Ruby on Rails
2. **Container as a Service (CaaS) – DockerHub**

OS-level virtualization

OS-level virtual image looks like real computers from the point of view of programs running on

- **Containers** ([LXC](#), [Solaris containers](#), [Docker](#)),
- **Zones** ([Solaris containers](#)), **Partitions**,
- **Virtual private servers** ([OpenVZ](#)), **Virtual environments** (VEs), **Virtual kernels** ([DragonFly BSD](#)), or
- All resources (connected devices, files and folders, network shares, CPU power, quantifiable hardware capabilities).
- Programs running inside of a container can only see the container's contents and devices assigned to the container.
- *Container* refer to OS-level virtualization systems, e.g. Microsoft's [Hyper-V](#) containers.

Container image

- Container is a running process interacting with its own private filesystem provided by a **Container image**.
- Container image runs an application with
 - the code or binary,
 - runtime dependencies, and
 - any other filesystem objects required.
- Docker is a platform for developers and sysadmins to **build, run, and share** applications with containers.

DockerHub is a CaaS

- Solomon Hykes and Sebastien Pahl
- OS-level virtualization to deliver software in packages called containers.
 - hosts the containers is called **Docker Engine**
- Docker can package an application and its dependencies in a virtual container that can run on any Linux server.
- Containers are isolated from one another and bundle their own software, libraries, and configuration files;
- All containers are run by a single operating system kernel and therefore use fewer resources than VMs.

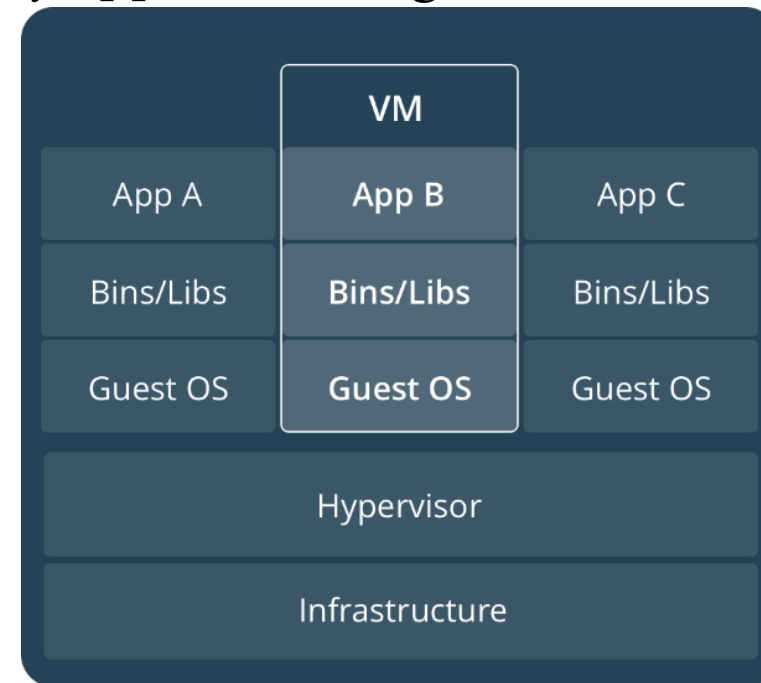
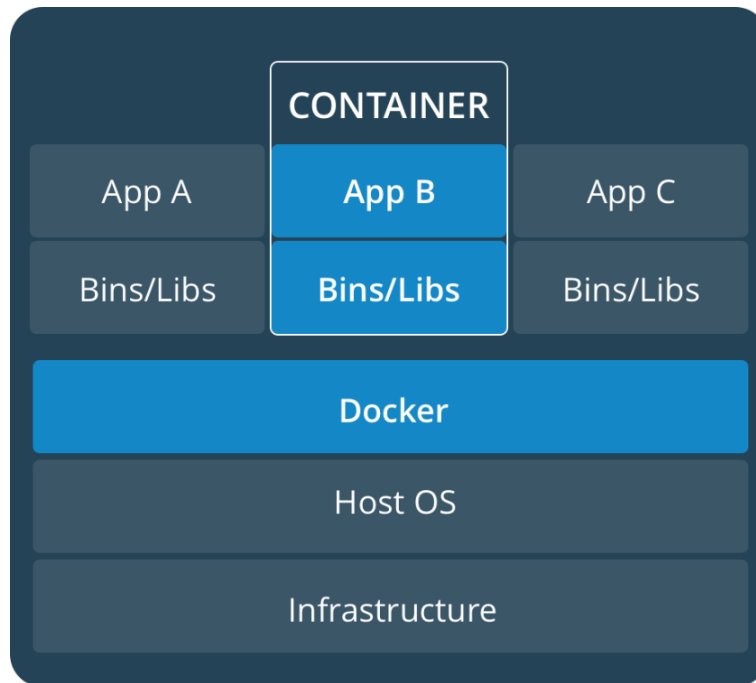
Docker Computational reproducibility

Docker implement and deployment of containers, includes:

- (1) cross-platform portability,
- (2) modular/component re-using and sharing,
- (3) Linux container (LXC) based OS level virtualization,
- (4) portable across platforms, and
- (5) archiving and versioning of container images.

Containers & Virtual machine (VM)

- Container runs *natively* on Linux and shares the kernel of the host machine with other containers. It runs a discrete process, taking memory like an executable.
- VM runs a full “guest OS” with hypervisor access to host resources. VM uses overhead resources that are not consumed by application logic.



Why PaaS or CaaS

Programming tools, Deployment tools, and Application hosting

- CaaS include stack of tools for deploying containers.
- CaaS do not include development tools.
- CaaS is a subset of PaaS functionality
- CaaS main use case is containerized applications
- CaaS is an incomplete form of a PaaS.
- PaaS and CaaS use-cases:
 - PaaS: integrated solution for develop and deploy applications
 - CaaS: easy to set-up and manage a container environment
- Benefit from a development at PaaS or not? Yes, then PaaS
- Pre-developed application for public deployment or not? Yes, then CaaS

PaaS and CaaS Hybrid

- PaaS-CaaS hybrid provides both development environment and deployment container
 - Amazon Elastic Container Service (ECS),
 - Ruby-on-Rails (EngineYard) and
 - Google App Engine etc.

<https://containerjournal.com/features/paas-vs-caas-wrong-question-ask/>

<https://aws.amazon.com/ecs/>

<https://www.engineyard.com/>

<https://cloud.google.com/appengine>

SaaS and Distributed Version Control (Facebook Testing, GitHub and Git-LFS)

1. IaaS and Hybrid Cloud
 - Orchestration & Virtualization: Eucalyptus & Amazon
 - Content Delivery Network (CDN): Facebook and Akamai
2. PaaS and Container as a Service (CaaS)
 - PaaS: Google App Engine (GAE) and Ruby on Rails
 - CaaS: DockerHub
3. **SaaS and Distributed Version Control (DVC)**
 - **SaaS: Facebook Testing (Infer and Sapienz)**
 - **DVC: GitHub and Git-LFS**

SaaS and Distributed Version Control

A. Scalable Facebook Testing:

- **Infern and**
- **Sapienz**
- **FBLearner**
- **Facebook's OneWorld platform**

B. Distributed Version Control:

- Version Control System (VCS), Central Version Control (CVC), Distributed Version Control (DVC)
- CVC v/s DVC
- Git, GitHub, GitHub Desktop (RaaS Client), and
- Git-LFS

Scalable Facebook Testing

- Scale means different things to different techniques to scale (or fail to scale) in different dimensions
- Scalability needed to assess the degree of deployability
- Challenges and Opportunities when deploying
 - Static analysis and Dynamic analysis at scale
- Facebook deploy two technologies
 - the Infer for static analysis and
 - the Sapienz dynamic analysis
 - Both are research-led start-up that was subsequently deployed at scale, impacting billions of people worldwide.

Harman, Mark, and Peter O'Hearn. "From start-ups to scale-ups: Opportunities and open problems for static and dynamic program analysis." *2018 IEEE 18th International Working Conference on Source Code Analysis and Manipulation (SCAM)*. IEEE, 2018.

Infer: Static Analysis at Scale

- At Facebook, Infer is a ‘deep’ static analysis tool applied to
 - Java, Objective C, and C++ code bases
- Infer at Facebook with acquisition of startup Monoidics in 2013
- Infer is open source and used at several companies
 - AWS, Mozilla, JD.com and Spotify
- Based on Separation Logic to scale algorithms for reasoning "states"
 - of a *store* and a *heap* about memory,
 - local (or stack-allocated) variables and dynamically-allocated,
 - implements a compositional program analysis: inter-procedural analysis,
 - follows pointer chains, safety of programs with embedded pointers,
 - scales to large code bases
- Result: 10s of 1000s bugs fix by Facebook developers before production.

Harman, Mark, and Peter O'Hearn. "From start-ups to scale-ups: Opportunities and open problems for static and dynamic program analysis." *2018 IEEE 18th International Working Conference on Source Code Analysis and Manipulation (SCAM)*. IEEE, 2018.

Infer: Static Analysis at Scale

- Facebook practices Continuous Development and Deployment i.e. Continuous Integration (CI) of “Diffs” (a code change) for code-review.
- Developers shares a single code base to alter and commit. Then, prepares a diff and submits it for the code review.
- Infer runs as a bot during diff time, writing comments for the developer and other human reviewers. Post-land incorporate the master build of the system.



Harman, Mark, and Peter O'Hearn. "From start-ups to scale-ups: Opportunities and open problems for static and dynamic program analysis." *2018 IEEE 18th International Working Conference on Source Code Analysis and Manipulation (SCAM)*. IEEE, 2018.

Sapienz: Dynamic Analysis at Scale

- Sapienz is deployed to run continuously testing the most recent internal builds of the Facebook apps.
- Sapienz is a multi-objective automated test case design system, that seeks to maximize fault revelation while minimizing the debug effort to fix.
- It uses
 - FBLearner (Facebook's Machine Learning infrastructure)
 - Facebook's OneWorld platform

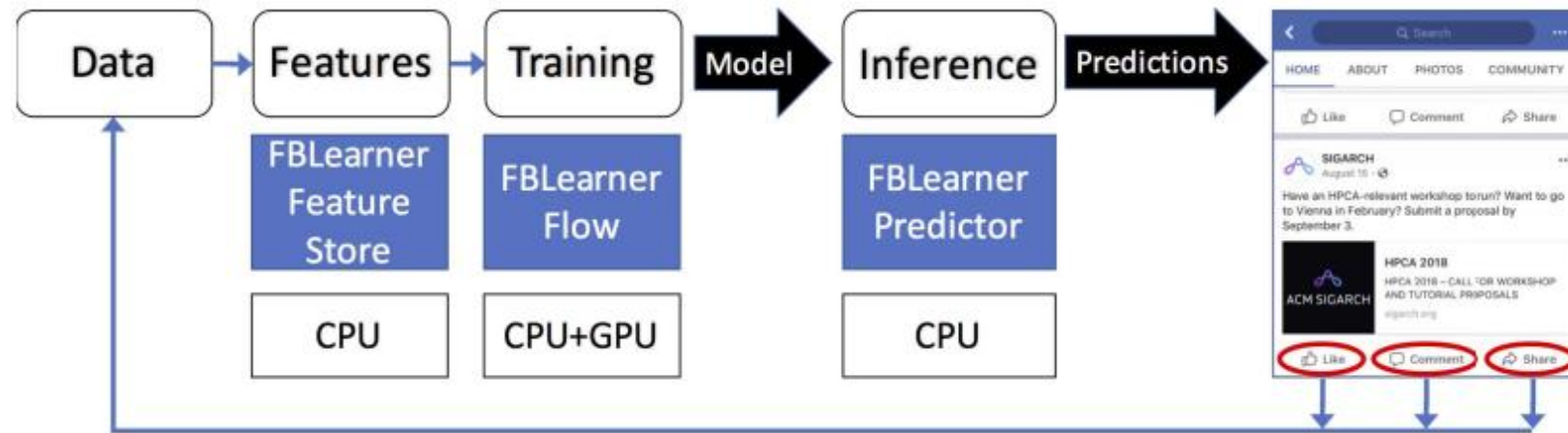
Sapienz: Dynamic Analysis at Scale

- FBLearner (Facebook's Machine Learning infrastructure)
 - Machine Learning (ML) perform at a scale, e.g., ML inference phase executions run into the tens of trillions per day.
 - FBLearner is a ML platform designed to support at a global scale
 - Sapienz is 1 of 100s services deployed on FBLearner framework
 - FBLearner Flow component in Sapienz
 - deploys detection of crashing behaviour directly into the work flow,
 - integrates with Phabricator for reporting and actioning fixes
 - corrects the failures detected
- Facebook's OneWorld platform
 - support scalable deployment on many of emulators, at a time
 - Sapienz deploys on 1000s of emulators to test the Android app alone

Harman, Mark, and Peter O'Hearn. "From start-ups to scale-ups: Opportunities and open problems for static and dynamic program analysis." *2018 IEEE 18th International Working Conference on Source Code Analysis and Manipulation (SCAM)*. IEEE, 2018.

FBLearner

ML-as-a-Service Inside Facebook: FBLearner Feature Store, FBLearner Flow, FBLearner Predictor.



MACHINE LEARNING ALGORITHMS LEVERAGED BY PRODUCT/SERVICE

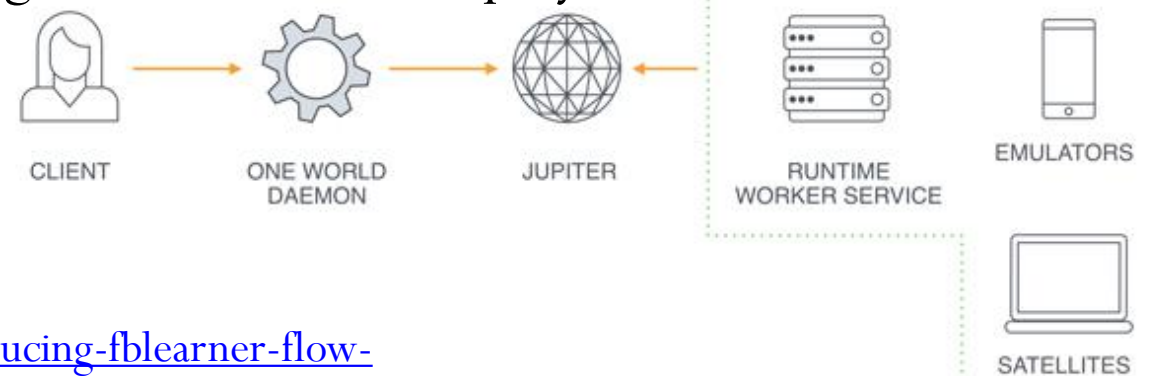
Models	Services
Support Vector Machines (SVM)	Facer (User Matching)
Gradient Boosted Decision Trees (GBDT)	Sigma
Multi-Layer Perceptron (MLP)	Ads, News Feed, Search, Sigma
Convolutional Neural Networks (CNN)	Lumos, Facer (Feature Extraction)
Recurrent Neural Networks (RNN)	Text Understanding, Translation, Speech Recognition

Hazelwood, Kim, et al. "Applied machine learning at facebook: A datacenter infrastructure perspective." *2018 IEEE International Symposium on High Performance Computer Architecture (HPCA)*. IEEE, 2018.

Facebook's OneWorld platform

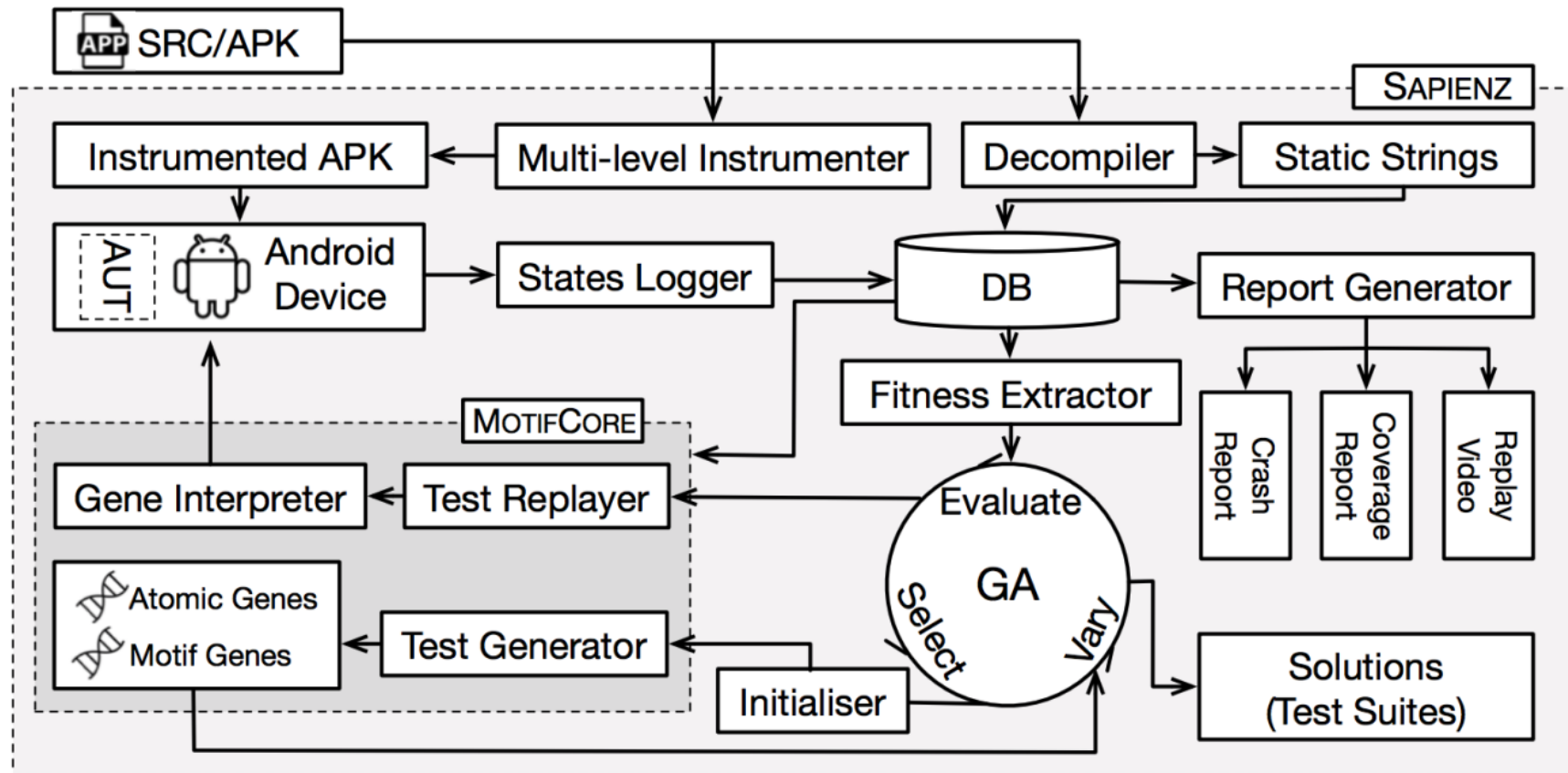
Support standard communication mechanisms like ADB (Android Debug Bridge) and Provide the illusion that remote devices are connected locally

- **Runtime worker service:** it manages the life cycle of the resource and responds to requests from clients to use its resources.
- **One World daemon:** it connect to remote resources with protocol to communicate with workers and sets up the environment.
- **Scheduler:** Uses Jupiter, a job-scheduling service at Facebook to match clients with workers' resources as per specified requirements.
- **Satellite:** it connect local resources to the global One World deployment.



Sapienz workflow

Sapienz Automated test design workflow



Harman, Mark, and Peter O'Hearn. "From start-ups to scale-ups: Opportunities and open problems for static and dynamic program analysis." *2018 IEEE 18th International Working Conference on Source Code Analysis and Manipulation (SCAM)*. IEEE, 2018.

SaaS and Distributed Version Control

A. Scalable Facebook Testing:

- Infern and
- Sapienz
- FBLearner
- Facebook's OneWorld platform

B. **Distributed Version Control:**

- **Version Control System (VCS), Central Version Control (CVC), Distributed Version Control (DVC)**
- **CVC v/s DVC**
- **Git, GitHub, GitHub Desktop (RaaS Client), and**
- **Git-LFS**

Version Control System (VCS)

- Version control is also known as
 - revision control,
 - source control, or
 - source code management
- It is responsible for managing changes to
 - computer programs,
 - documents,
 - large web sites, or
 - other collections of information.
- It is a component of Software Configuration Management (SCM).
- VCS are stand-alone applications.

Centralised Version Control (CVC)

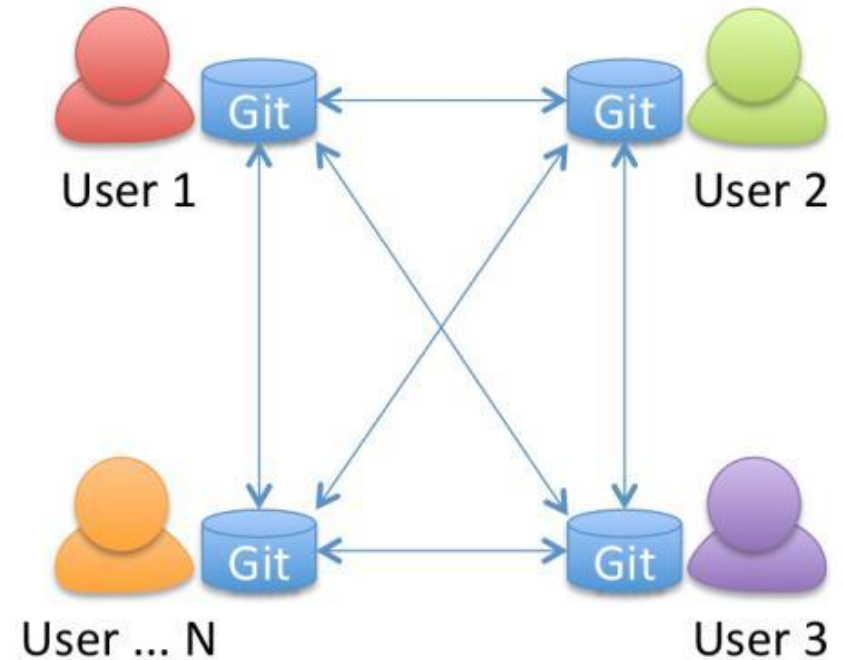
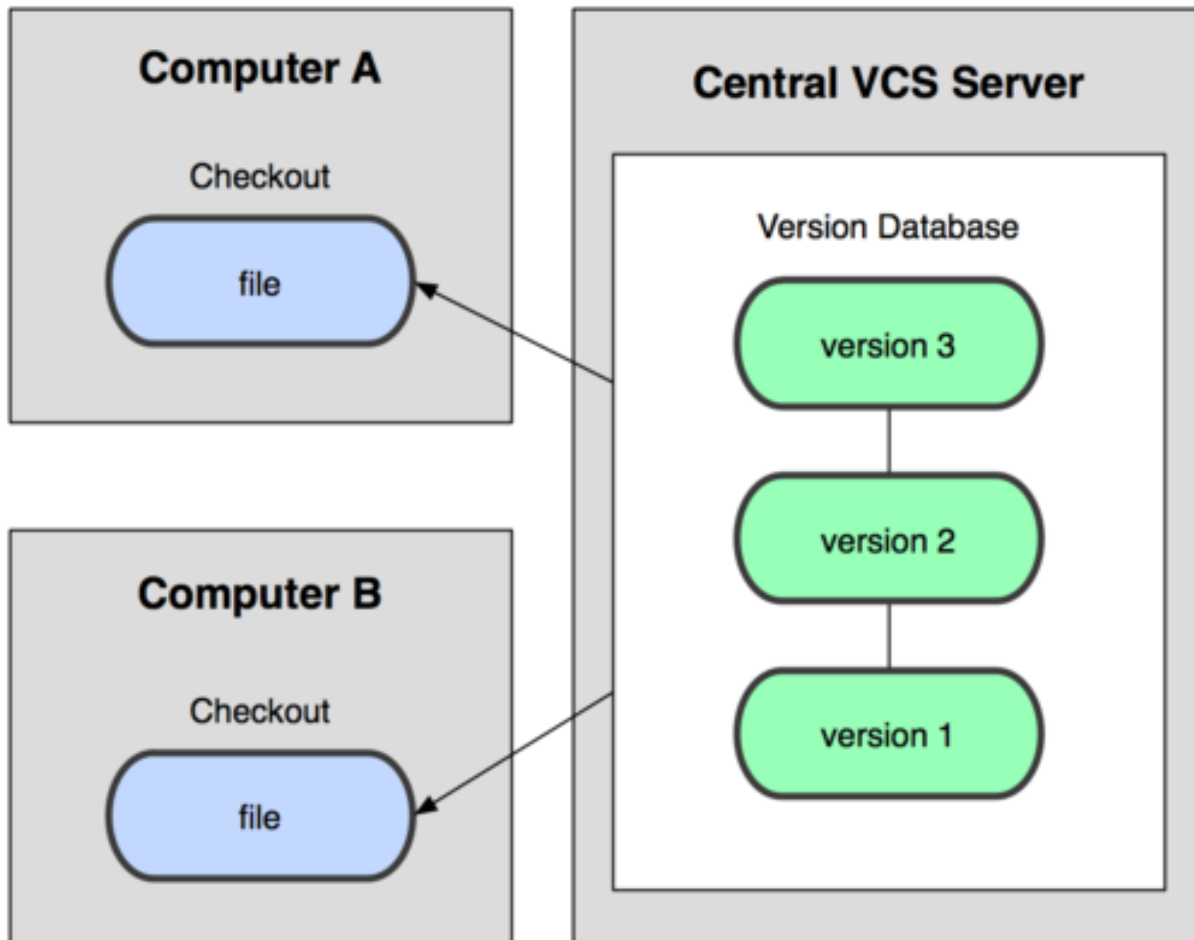
- Single server holds the code base
- Clients access the server with check-in/check-outs
- CVC available are
 - CVS,
 - Subversion,
 - Visual Source Safe.
- Pro: Simple to manage a single server
- Con: Single point of failure.

Distributed Version Control (DVC)

- aka, Distributed Revision Control
- It is a form of version control that complete codebase, including its full history, is mirrored on every developer's computer. Enables
 - automatic management branching and merging,
 - speeds up most operations (except pushing and pulling),
 - improves the ability to work offline and
 - does not rely on a single location for backups.
- Each client holds a copy of the code base.
- Code is shared between clients by push/pulls
 - Advantages: Many operations, no single point of failure
 - Disadvantages: Complex to manage

https://en.wikipedia.org/wiki/Distributed_version_control

CVC v/s DVC



Git

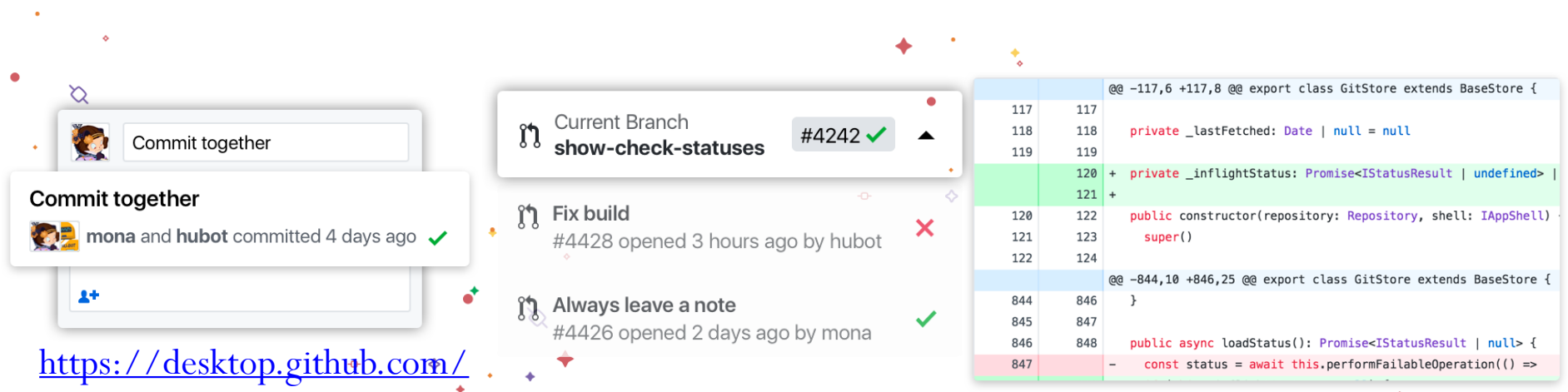
- **Git** is a distributed version-control system.
- It track changes in source code files during software development.
- It is designed for coordinating work among programmers,
- Its goal include speed, data integrity, and support for distributed, non-linear workflows.
- Git was created by Linus Torvalds in 2005 for development of the Linux kernel.
- Every Git directory on every developer's computer is a repository, and independent of network or central server.

GitHub

- **GitHub** provides web-hosting functionalities for
 - Software development,
 - Distributed Version Control, and
 - Source Code Management (SCM).
- GitHub features
 - bug tracking, continuous integration, change management,
 - feature requests, task management, wikis,
 - *Branching, Merging, Fetch, and Commit* of new *push* files
 - *pull* or *clone* of whole existing repository or its file(s)
- January 2020, GitHub reports
 - over 40 million users and
 - more than 190 million repositories (28 million+ public repositories),
 - making it the largest host of source code in the world.

GitHub Desktop (RaaS Client)

- It simplifies development workflow by abstracting Git.
 - Attribute commits with collaborators: co-authors
 - Checkout branches with pull requests and view Commit statuses
 - Syntax highlighted diffs: highlighting when viewing diffs
 - Expanded image diff support: compare changed images
 - Extensive editor & shell integrations: use favorite editor or shell
 - Open-source Community support: contribute and collaboration



GitHub Desktop (RaaS Client)

The screenshot displays the GitHub Desktop application window. The top bar includes a menu (File, Edit, View, Repository, Branch, Help) and window controls. Below the menu, the 'Current repository' is set to 'desktop', the 'Current branch' is 'esc-pr' (commit #3972), and the 'Fetch origin' button is visible, indicating the last fetch was 2 minutes ago.

The main area is divided into two panes. The left pane shows the 'History' tab with a list of commits:

- Appease linter (iAmWillShepherd committed a day ago)
- Add event handler to dropdown compon... (iAmWillShepherd and Markus Olsson co...)
- Move escape behavior to correct compo... (iAmWillShepherd and Markus Olsson co...)
- Remove event handler from the branches.. (iAmWillShepherd and Markus Olsson co...)
- Merge branch 'master' into esc-pr (iAmWillShepherd committed a day ago)
- Merge pull request #4044 from desktop/... (Neha Batra committed a day ago)
- Merge pull request #4070 from desktop/... (Brendan Forster committed 2 days ago)
- bump to beta3 (Brendan Forster committed 2 days ago)
- Merge pull request #4057 from desktop/... (Brendan Forster committed 2 days ago)
- Merge pull request #4067 from desktop/... (Brendan Forster committed 2 days ago)
- Release to 1.1.0-beta2 (Neha Batra committed 2 days ago)

The right pane shows the diff for the selected commit 'Add event handler to dropdown component'. It indicates that iAmWillShepherd and Markus Olsson committed this change (c79e71c), which modified 1 file. The diff shows changes to 'app\src\ui\toolbar\dropdown.tsx'.

The diff view includes a table of line changes:

Line	Original	Modified
145		145
146		146
147		147
148		148
149		149
150		150
151		151
152	148	152
153	149	153
154	150	154
249		253
250		254
251		255
256		256
257		257
258		258

The diff content shows the following code changes:

```
@@ -145,6 +145,10 @@ export class ToolbarDropdown extends React.Component<
  this.state = { clientRect: null }
}

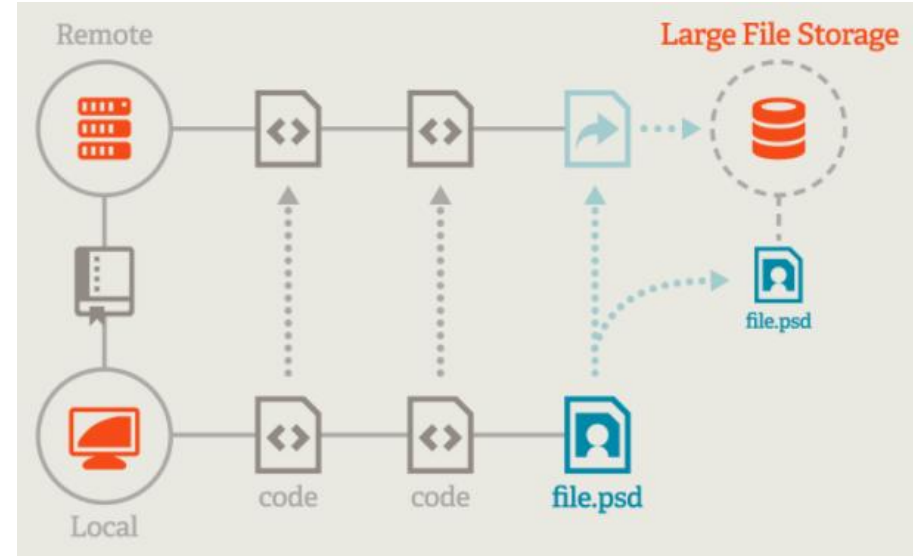
+ private get isOpen() {
+   return this.props.dropdownState === 'open'
+ }
+
+ private dropdownIcon(state: DropdownState): OcticonSymbol {
+   // @TODO: Remake triangle octicon in a 12px version,
+   // right now it's scaled badly on normal dpi monitors.
+ }

@@ -249,6 +253,13 @@ export class ToolbarDropdown extends React.Component<
  }
  }
  }

+ private onFoldoutKeyDown = (event: React.KeyboardEvent<HTMLElement>) => {
+   if (!event.defaultPrevented && this.isOpen && event.key === 'Escape') {
+     event.preventDefault()
+   }
+ }
```


Git-LFS

- Git Large File Storage (LFS)
 - audio,
 - videos,
 - datasets, and
 - graphics,
- Git-LFS replaces large files with text pointers inside Git
- Storing the file contents on a remote server like
 - GitHub.com or GitHub Enterprise
 - *git lfs* install
 - *git add* file.psd
 - *git commit -m* "Add design file"
 - *git push* origin master



תודה רבה

Hebrew

Ευχαριστώ

Greek

Спасибо

Russian

Danke

German

Merci

French

धन्यवादः

Sanskrit

நன்றி

Tamil

شكراً

Arabic

ಧನ್ಯವಾದಗಳು

Kannada

Thank You

English

നന്നി

Malayalam

Grazie

Italian

ధన్యవాదాలు

Telugu

આભાર

Gujarati

多謝

Traditional Chinese

Gracias

Spanish

ਧੰਨਵਾਦ

Punjabi

धन्यवाद

Hindi & Marathi

多谢

Simplified Chinese

<https://sites.google.com/site/animeshchaturvedi07>

Obrigado

Portuguese

ありがとうございました

Japanese

ขอบคุณ

Thai

감사합니다

Korean